

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 2

по курсу «Алгоритмы компьютерной графики»

«Модельно-видовые и проективные преобразования»

Студент группы ИУ9-42Б Лавров Р. Д.

Преподаватель Цалкович П. А.

Москва 2025

1 Задача

1. Определить куб в качестве модели объекта сцены.

2. Определить преобразования, позволяющие получить заданный вид проекции (в соответствии с вариантом). Для демонстрации проекции добавить в сцену куб (в стандартной ориентации, не изменяемой при модельно-видовых преобразованиях основного объекта).

3. Реализовать изменение ориентации и размеров объекта (навигацию камеры) с помощью модельно-видовых преобразований (без gluLookAt). Управление производится интерактивно с помощью клавиатуры и/или мыши.

4. Предусмотреть возможность переключения между каркасным и твердотельным отображением модели (glFrontFace/ glPolygonMode).

2 Основная теория

Ключевым элементом работы является применение матричных преобразований для получения требуемого вида проекции. В сцене присутствуют два куба с различными наборами трансформаций.

2.1 Матрица трёхточечной проекции

В работе используется следующая матрица трёхточечной проекции:

$$\mathbf{M}_{\text{3P}} = \begin{bmatrix} 0.87 & -0.09 & 0.98 & 0.49 \\ 0 & 0.98 & 0.35 & 0.17 \\ 0.5 & 0.15 & -1.7 & -0.85 \\ 0 & 0 & 1 & 2 \end{bmatrix}$$

Данная матрица задаёт параметры проекции, которые имитируют перспективное искажение с тремя точками схода. Она применяется для преобразования координат трёхмерного объекта в двухмерное пространство экрана.

Вспомогательный куб демонстрирует базовую проекцию без дополнительных преобразований:

- Смещается в левый верхний угол командой $\mathrm{glTranslatef}(0.6,\ 0.6,\ 0)$.
- К нему применяется матрица трёхточечной проекции $\mathbf{M}_{\text{3Р}}$.

Основной куб подвергается последовательным преобразованиям:

- Масштабирование ($\mathrm{glScalef}$) — изменяет линейные размеры объекта относительно его центра.
- Проекция ($\mathrm{glMultMatrixf}$) — применяет матрицу трёхточечной проекции $\mathbf{M}_{\text{3Р}}$.
- Поворот ($\mathrm{glRotatef}$) — вращает куб вокруг осей X и Y на углы, задаваемые пользователем.

В OpenGL матричные преобразования применяются в порядке, обратном их вызову. Таким образом, результат преобразований для куба имеет вид:

$$\mathbf{P}_{\text{итог}} = \mathbf{T}_{\text{масштаб}} \cdot \mathbf{M}_{\text{3Р}} \cdot \mathbf{R}_x \cdot \mathbf{R}_y$$

где $\mathbf{R}_x$ и $\mathbf{R}_y$ — матрицы поворота вокруг осей X и Y соответственно.

2.2 Интерактивное управление и визуальные режимы

Взаимодействие с пользователем реализовано через клавиатурный ввод с использованием $\mathrm{key_callback}$:

Управление параметрами объекта:

- Клавиши-стрелки изменяют углы поворота, обеспечивая вращение основного куба вокруг осей координат.
- Клавиши +/- регулируют коэффициент масштабирования, увеличивая или уменьшая размер объекта.
- Глобальные переменные обновляются в обработчике и используются при каждой перерисовке кадра для применения актуальных трансформаций.

Переключение режимов визуализации осуществляется по нажатию клавиши ’M’:

- Переменная wireframe инвертирует своё значение.
- Функция glPolygonMode() с параметрами GL_FRONT_AND_BACK, GL_LINE включает каркасный режим, при котором отображаются только рёбра куба.
- Режим GL_FRONT_AND_BACK, GL_FILL возвращает твердотельное отображение с закрашенными гранями.

2.3 Структура основного цикла визуализации

Функция display() формирует каждый кадр сцены:

1. Очистка буферов цвета и глубины для корректной перерисовки.
2. Сохранение текущего состояния матрицы проекции (glPushMatrix).
3. Последовательная отрисовка вспомогательного и основного кубов с применением соответствующих трансформаций.
4. Восстановление предыдущего состояния матрицы (glPopMatrix).

Главный цикл приложения обеспечивает непрерывную работу: обрабатывает события, вызывает отрисовку и обновляет содержимое окна.

3 Практическая реализация

Листинг 1: Основной код

```
import glfw
from OpenGL.GL import *
import numpy as np

angle_x, angle_y = 0, 0
wireframe = False
scale  = 1.0

```

```
def three_point_projection():
    M_3P = [
            0.87,  -0.09,  0.98,  0.49,
            0,  0.98,  0.35,  0.17,
            0.5,  0.15,  -1.7,  -0.85,
            0,  0,  1,  2,
        ]
    return M_3P

def main():
    if not glfw.init():
        return
    window = glfw.create_window(800, 800, "lab2", None, None)
    if not window:
        glfw.terminate()
        return
    glfw.make_context_current(window)
    glfw.set_key_callback(window, key_callback)

    glEnable(GL_DEPTH_TEST)
    glClearColor(0.2,  0.2,  0.2,  1.0)

    while not glfw.window_should_close(window):
        display()
        glfw.swap_buffers(window)
        glfw.poll_events()

    glfw.destroy_window(window)
    glfw.terminate()

def draw_cube():
    face_colors = [
        [0.8,  0.0,  0.0],
        [0.0,  0.8,  0.0],
        [0.0,  0.0,  0.8],
        [0.8,  0.8,  0.0],
        [0.8,  0.0,  0.8],
        [0.0,  0.8,  0.8],
    ]

    vertices = [
        [-0.3,  -0.3,  -0.3],  [0.3,  -0.3,  -0.3],  [0.3,  0.3,  -0.3],  [-0.3,  0.3,  -0.3],
        [-0.3,  -0.3,  0.3],  [0.3,  -0.3,  0.3],  [0.3,  0.3,  0.3],  [-0.3,  0.3,  0.3]
    ]
    faces = [
        [0,  1,  2,  3],  [4,  5,  6,  7],  [0,  4,  7,  3],
```

```
55            [1, 5, 6, 2], [3, 2, 6, 7], [0, 1, 5, 4]
56        ]
57
58        glPolygonMode(GL_FRONT_AND_BACK, GL_LINE if wireframe else GL_FILL)
59
60        for i, face in enumerate(faces):
61            glBegin(GL_QUADS)
62            glColor3fv(face_colors[i])
63            for vertex in face:
64                glVertex3fv(vertices[vertex])
65            glEnd()
66
67    def display():
68        global angle_x, angle_y, scale
69
70        glLoadIdentity()
71        glClear(GL_COLOR_BUFFER_BIT)
72        glClear(GL_DEPTH_BUFFER_BIT)
73        glMatrixMode(GL_PROJECTION)
74        glPushMatrix()
75
76        glTranslatef(0.6, 0.6, 0)
77        glMultMatrixf(three_point_projection())
78        draw_cube()
79        glPopMatrix()
80
81        glTranslatef(-0.3, 0, 0)
82        glScalef(scale, scale, scale)
83        glMultMatrixf(three_point_projection())
84        glRotatef(angle_x, 1, 0, 0)
85        glRotatef(angle_y, 0, 1, 0)
86        draw_cube()
87
88
89    def key_callback(window, key, scancode, action, mods):
90        global angle_x, angle_y, wireframe, scale
91        if action in [glfw.PRESS, glfw.REPEAT]:
92            if key == glfw.KEY_RIGHT:
93                angle_y -= 3
94            if key == glfw.KEY_LEFT:
95                angle_y += 3
96            if key == glfw.KEY_UP:
97                angle_x -= 3
98            if key == glfw.KEY_DOWN:
99                angle_x += 3
100           if key == glfw.KEY_KP_ADD or key == glfw.KEY_EQUAL:
```

```
            scale += 0.1
        if key == glfw.KEY_KP_SUBTRACT or key == glfw.KEY_MINUS:
            scale = max(0.1, scale - 0.1)
        if key == glfw.KEY_M:
            wireframe = not wireframe

if __name__ == "__main__":
    main()
```

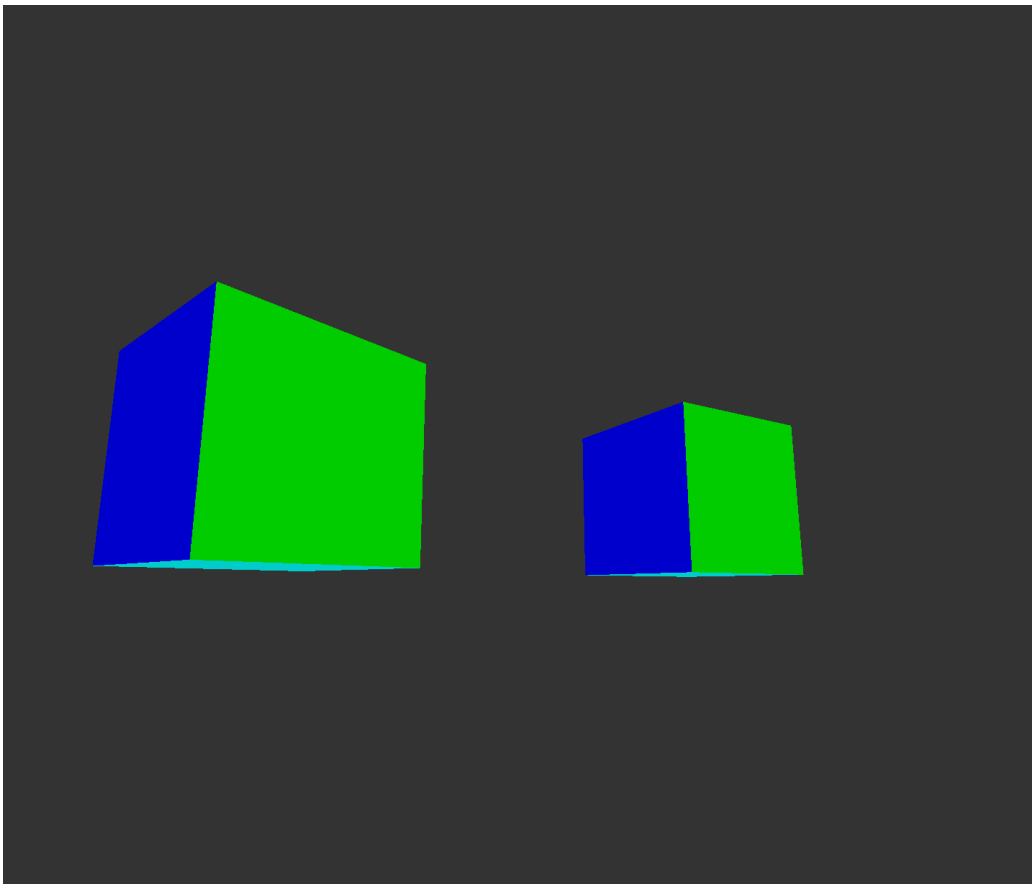


Рис. 1 — Результат работы

4 Вывод

В ходе выполнения лабораторной работы была разработана интерактивная трёхмерная сцена с использованием библиотек GLFW и OpenGL. Создана модель куба, позволяющая наглядно отслеживать его ориентацию в пространстве.

Реализована матрица трёхточечной проекции обеспечивающая перспективное искажение с тремя точками схода.

Освоены принципы модельно-видовых преобразований: поворот, масштабирование и применение проекционной матрицы. Реализовано интерактивное управление объектом с клавиатуры: стрелки управляют поворотом куба, клавиши ’+’ и ’-’ регулируют масштаб. По нажатию клавиши ’M’ осуществляется переключение между каркасным и твердотельным режимами отображения.